“Question Bank Management System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

J.ADVAITH :2520030524
O.SAI SUJITH :2520030129
P.SHASHE PREETHAM :2520030534

Team Number: 23

Section: 6

Under the Guidance of

Dr. V. Sireesha
Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Question Bank Management System"** is a record of Bonafide work of **J. ADVAITH – 2520030524, O. SAI SUJITH – 2520030129, P. SHASHE PREETHAM – 2520030534** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

J. Advaith – 2520030524
O. Sai Sujith – 2520030129
P. Shashe Preetham – 2520030534

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Question Bank Management System"** is a bonafide work done and submitted by **J. ADVAITH – 2520030524, O. SAI SUJITH – 2520030129, P. SHASHE PREETHAM – 2520030534** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. V. Sireesha)

Signature of the HOD
(Dr. P. Venkateshwara Rao)

Signature of the External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Dr. V. Sireesha**, Professor, Department of Computer Science and Engineering, for her valuable guidance, constant encouragement and insightful suggestions throughout the course of this project. Her support helped us understand the practical application of advanced Data Structures and Algorithms and complete the Question Bank Management System successfully.

We are also thankful to **Dr. P. Venkateshwara Rao**, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and a supportive academic environment that made this work possible. We extend our thanks to all the faculty members of the department for their cooperation during the Data Structures and Algorithms -3 (25CS2103E) course.

Finally, we thank our friends and family for their encouragement and support, and our team members for their dedication and teamwork in designing, implementing and documenting this project.

J. Advaith – 2520030524
O. Sai Sujith – 2520030129
P. Shashe Preetham – 2520030534

Abstract

Educational institutions accumulate questions across semesters, subjects, units and difficulty levels, but these questions are often scattered across files, folders and printed sheets. Faculty must search manually, repeated or near-duplicate questions slip into question papers, and there is little visibility into how often a question has already been used. To address this, our project focuses on the Design and Development of a Question Bank Management System (QBMS), a DSA-3 driven Java application for intelligent question retrieval and paper generation. The core aim is to treat the question bank as a text-search and optimisation problem and to make the algorithms, rather than a database, the main product.

The system stores questions in structured text files, categorised by subject, unit, difficulty, course outcome, marks and tags, and loads them into custom in-memory data structures. Exact keyword search is provided by the Knuth-Morris-Pratt and Rabin-Karp algorithms, and multi-keyword search by the Aho-Corasick automaton. The Wagner-Fischer edit distance supports fuzzy retrieval for misspelled queries, while Suffix Array with the LCP array detects highly similar or repeated questions. A 0/1 Knapsack dynamic-programming module generates a question paper for a chosen subject, number of questions and target total marks, penalising overused questions using the recorded usage history.

The application follows a layered architecture consisting of an HTML, CSS and JavaScript front end with Questions, Search, Generate Paper, Usage History, Add Question, Benchmark and Users tabs, a Java back-end API layer, a service layer, an algorithm layer and a file-based data layer. Admin and Faculty roles control access to the features. The algorithms are implemented manually without external libraries, and every search reports the algorithm used, matched questions, comparison count, execution time and complexity. QBMS therefore serves as a practical tool that reduces manual searching and improves consistency in assessment preparation, and as an educational platform for understanding string algorithms, dynamic programming and complexity analysis.

Table of Contents

			Page No.
	Acknowledgement		iv
	Abstract		v
	Table of Contents		vi
	List of Figures		vii
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Existing System versus Proposed System 1.6 Relevance to DSA-3 1.7 Approach Towards the Problem	1-3
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements 2.4 User Roles and Use Cases	4-5
3	System Design	3.1 High-Level Architecture Diagram 3.2 Description of Each Layer 3.3 Data Flow Architecture 3.4 Algorithm Selection Architecture 3.5 Component Interaction 3.6 Data Storage Design 3.7 Class Design 3.8 Security and Session Architecture 3.9 Design Principles 3.10 Overall System Workflow	6-10
4	Implementation	4.1 Module-wise Implementation 4.2 Front-end Logic (UI rendering, state management) 4.3 API Integration 4.4 Data Processing Flow 4.5 Error Handling 4.6 Implementation Summary	11-22
5	Results	5.1 Functional Results 5.2 User Interface Results 5.3 Algorithm Complexity Comparison 5.4 Test Results 5.5 Overall Result	23-25
6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System	26
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development	27
	References	Books, papers, APIs and documentation sites used	28

List of Figures

1.1	Overall architecture of the Question Bank Management System	1
1.2	Proposed system pipeline from question loading to paper export	3
2.1	Use case diagram for Admin and Faculty roles	5
3.1	High-level layered architecture diagram	6
3.2	Search request architecture and algorithm routing	7
3.3	Data flow architecture of QBMS	8
3.4	Component interaction diagram	8
3.5	File-based data storage design	9
3.6	Class diagram of the major model and algorithm classes	9
3.7	Overall system workflow	10
4.1	Server start-up and request handler registration	11
4.2	Question object converted into searchable text	11
4.3	Source code of the SearchResult model class	12
4.4	Login and authentication flow	13
4.5	Add-question flow with validation and similarity check	13
4.6	Search service flow	14
4.7	Aho-Corasick trie with failure links (patterns: he, hers, his, she)	14
4.8	Source code of the AhoCorasick algorithm class	15
4.9	Flowchart of the KMP algorithm	15
4.10	Flowchart of the Rabin-Karp algorithm	16
4.11	Flowchart of Wagner-Fischer edit distance	17
4.12	Suffix Array and LCP based similarity detection	18
4.13	Paper generation using 0/1 Knapsack dynamic programming	19
4.14	Usage tracking flow	20
4.15	Front-end styling code (glass-morphism theme) in index.html	21
5.1	Login page of QBMS	23
5.2	Dashboard with statistics and the Questions tab	24
5.3	Generate Paper tab with subject, question count and target marks	24

1. INTRODUCTION

1.1 Overview

The **Question Bank Management System (QBMS)** is a Java-based academic application that turns scattered question material into a structured, searchable and reusable question bank. Educational institutions accumulate hundreds of questions across semesters, subjects, units and difficulty levels. Over time these questions end up in different files, folders and printed sheets, which makes it slow for faculty to find a suitable question and easy to repeat a question that was already used in an earlier paper.

QBMS addresses this by treating the question bank as a text-retrieval and optimisation problem rather than a database problem. Questions are stored in readable, structured text files and loaded into custom in-memory data structures. Advanced string-matching algorithms such as KMP, Rabin-Karp and Aho-Corasick perform exact and multi-keyword retrieval; Wagner-Fischer edit distance supports fuzzy retrieval for misspelled queries; Suffix Array with the LCP array detects highly similar questions; and a 0/1 Knapsack dynamic-programming module selects an optimal set of questions for a question paper while penalising questions that have been overused. The overall structure of the system is shown in Fig. 1.1.

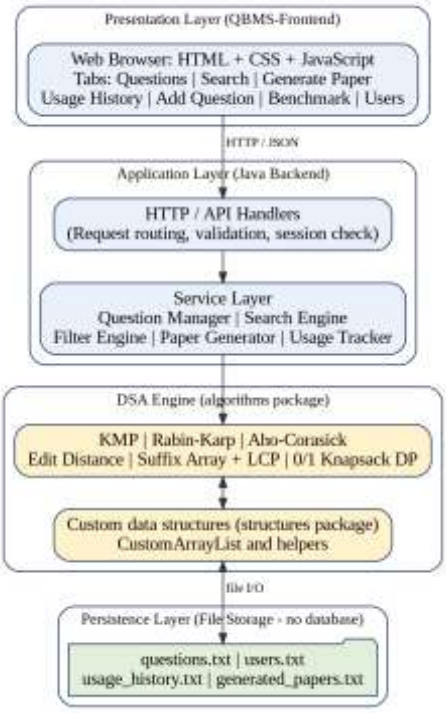


Fig. 1.1 Overall architecture of the Question Bank Management System

The system is delivered as a web-style application: an HTML, CSS and JavaScript front end (the QBMS-Frontend project) communicates with a Java back end whose algorithm package is implemented manually, without external algorithm libraries. Persistence is provided entirely by files, so the project can be deployed with only a Java runtime and its data folder. The user interface, shown in Chapter 5, offers tabs for Questions, Search, Generate Paper, Usage History, Add Question, Benchmark and Users, together with live statistics for total questions, subjects, papers generated and usage records.

1.2 Problem Statement

Faculty members routinely need to select questions that match a given subject, unit, difficulty level and course outcome (CO) while keeping the total marks balanced. In practice the academic workflow is simple, but the data problem is not. The main pain points observed are:

- Questions are scattered across files, folders and printed sheets.
- Finding a suitable question requires repeated manual searching.
- Similar or repeated questions can enter different papers without being noticed.
- Difficulty, unit and CO balance is hard to maintain manually.
- There is little visibility into the previous usage of a question.

These problems lead to higher preparation time, repeated questions, inconsistent paper quality, poor reuse of existing material and more effort for the faculty during assessment design. A plain keyword search is also not enough: a teacher who types "knuth moris pratt" should still find the KMP question, and two questions worded differently may in fact test the same concept.

Question banks are fundamentally text-search and optimisation problems. QBMS therefore makes the algorithms the main product instead of building a conventional CRUD-plus-database application, and uses the algorithms to give fast, tolerant and duplicate-aware retrieval together with constraint-based paper generation.

1.3 Scope of the Project

The scope of QBMS is to develop a DSA-3 driven question retrieval and paper-generation application that works entirely on file-based storage. The project covers the following areas:

1. **Question management** - add, edit, delete and view questions with subject, unit, difficulty, CO, marks and tags.
2. **Advanced exact search** - KMP and Rabin-Karp for single-pattern search and Aho-Corasick for simultaneous multi-keyword search.
3. **Fuzzy search** - Wagner-Fischer edit distance to suggest close matches for misspelled queries.
4. **Similarity detection** - Suffix Array and LCP to flag highly similar or repeated question text.
5. **Smart filters** - filtering by subject, unit, difficulty, CO and marks.
6. **Paper generation** - 0/1 Knapsack dynamic programming to select a question set that meets a target total marks while penalising overused questions.
7. **Usage tracking** - recording every question used in a generated paper in a usage-history file.
8. **Role-based access** - Admin and Faculty accounts maintained in a users file.
9. **Benchmarking** - comparing the number of character comparisons and execution time of the algorithms on the same question bank.

QBMS is an academic project. It is not intended to replace an institutional examination system, and it does not use a database server, network-scale indexing or production-grade authentication.

1.4 Objectives

The main objectives of the project are expressed as measurable DSA work:

- **Fast retrieval** - search a large question file efficiently using advanced string-matching algorithms.
- **Fuzzy search** - handle spelling variation using edit-distance based matching.
- **Similarity** - detect closely related questions using suffix-based text analysis.
- **Smart selection** - generate a balanced set of questions from constraints using dynamic programming.
- **No database** - keep persistence simple with readable, structured text files.
- **Complexity awareness** - compare the algorithms using their time and space complexity and a built-in benchmark.
- **Extensibility** - keep the algorithm layer modular so that new algorithms can be added easily.

1.5 Existing System versus Proposed System

Aspect	Typical / Existing Approach	Proposed QBMS (DSA-3 Implementation)
Storage	Database-centric CRUD application or loose document files	Java application with structured, human-readable file storage
Search	Basic keyword / linear search	KMP and Rabin-Karp for efficient text search; Aho-Corasick for multi-keyword search
Misspelled queries	Not handled	Edit distance gives fuzzy query matching
Duplicate questions	Checked manually	Suffix Array + LCP similarity analysis
Paper preparation	Manual filtering and selection	Algorithm-driven selection using 0/1 Knapsack DP
Question reuse	Not tracked or checked by hand	Usage history stored and used to penalise overused questions

1.6 Relevance to Data Structures and Algorithms - 3

The project directly uses the advanced algorithm families covered in the DSA-3 course (25CS2103E), as summarised below.

Course Module	Concept	Use in QBMS
Module 2 - String Algorithms	KMP, Rabin-Karp, Aho-Corasick, suffix structures	Question retrieval, multi-keyword search and similarity detection
Module 3 - Dynamic Programming	Wagner-Fischer edit distance, 0/1 Knapsack	Fuzzy matching and constraint-based paper generation
Module 5 - Complexity and Approximation	Time/space analysis, approximation ideas	Complexity comparison of search and selection costs; benchmark tab
Module 6 - Randomised Algorithms	Randomised hashing	Hash-assisted search in Rabin-Karp with a prime modulus

1.7 Approach Towards the Problem

The workflow of QBMS is designed around queries rather than around tables. Fig. 1.2 shows the six stages followed by every request.

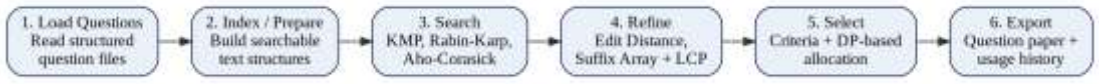


Fig. 1.2 Proposed system pipeline from question loading to paper export

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The functional requirements define the operations and services that QBMS is expected to provide.

ID	Requirement	Description
FR1	User authentication	The system shall allow Admin and Faculty users to sign in with valid credentials before accessing protected features.
FR2	Question management	The system shall allow questions with subject, unit, difficulty, CO, marks, text and tags to be added, edited, deleted and viewed.
FR3	Filtering	The system shall filter questions by subject, unit, difficulty, CO and marks.
FR4	Exact search	The system shall support exact keyword searching using KMP and Rabin-Karp.
FR5	Multi-keyword search	The system shall search several keywords in a single pass using Aho-Corasick.
FR6	Fuzzy search	The system shall suggest close matches for misspelled queries using edit distance.
FR7	Similarity check	The system shall flag highly similar or repeated questions using Suffix Array and LCP.
FR8	Paper generation	The system shall generate a question paper for a chosen subject, number of questions and target total marks using 0/1 Knapsack DP.
FR9	Usage tracking	The system shall record every question used in a generated paper and show the usage history.
FR10	Search report	The system shall report the algorithm used, matched question IDs, comparisons, execution time and complexity for every search.
FR11	Benchmarking	The system shall let the user compare algorithms on the same question bank.
FR12	User management	The Admin shall be able to view and manage users stored in the users file.

2.2 Non-Functional Requirements

1. **Performance** - search and selection should remain responsive for the intended question-bank size, and exact search should outperform naive matching for suitable inputs.
2. **Usability** - the interface should provide simple tabs, filters, forms and readable result cards, including a light/dark theme.
3. **Reliability** - invalid inputs, empty queries and missing records must be handled without terminating the application.
4. **Maintainability** - the front end, API handlers, services, algorithms, models and file storage must remain clearly separated.
5. **Portability** - the application should run on any standard Java environment without a database server or heavy framework.

6. **Scalability** - the architecture should allow more questions, algorithms and filters to be added later.
7. **Security** - protected features must require authentication, and passwords should not be stored in plain form.
8. **Extensibility** - new algorithms should be pluggable into the algorithm layer without major changes elsewhere.

2.3 Hardware and Software Requirements

Category	Requirement
Processor	Any modern dual-core processor (Intel Core i3 / AMD Ryzen 3 or higher)
Memory	4 GB RAM minimum (8 GB recommended)
Storage	100 MB free disk space for the application and data files
Operating system	Windows 10/11, Linux or macOS
Back end	Java Development Kit (JDK) 17 or later; Java File I/O; manually implemented algorithms
Front end	HTML5, CSS3 and JavaScript (no external UI framework)
Data storage	Structured text files (questions.txt, users.txt, usage_history.txt, generated_papers.txt)
Tools	Visual Studio Code, command-line Java compiler, any modern web browser

2.4 User Roles and Use Cases

QBMS supports two roles. The **Admin** has complete access, including benchmarking and user management. The **Faculty** role can manage and search questions and generate papers. The interaction of both roles with the system is shown in Fig. 2.1.

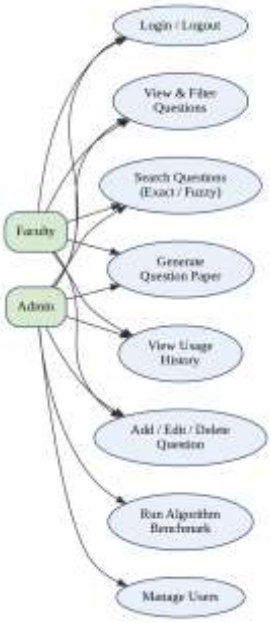


Fig. 2.1 Use case diagram for Admin and Faculty roles

3. SYSTEM DESIGN

The system design of QBMS defines how the front end, the Java back end, the service modules, the algorithm layer and the file storage interact. The architecture keeps the user interface, application logic, DSA implementations and data management logically separated, so that each part can be developed, tested and replaced independently. It also allows different algorithms to be applied according to the type of operation requested by the user.

3.1 High-Level Architecture Diagram

The architecture follows a layered approach, as shown in Fig. 3.1. A request begins at the presentation layer and is sent to the Java back end through HTTP. The API layer routes it to the appropriate service, which validates the input and invokes either a data-management operation or the required algorithm. The result is returned through the same layers and displayed to the user.

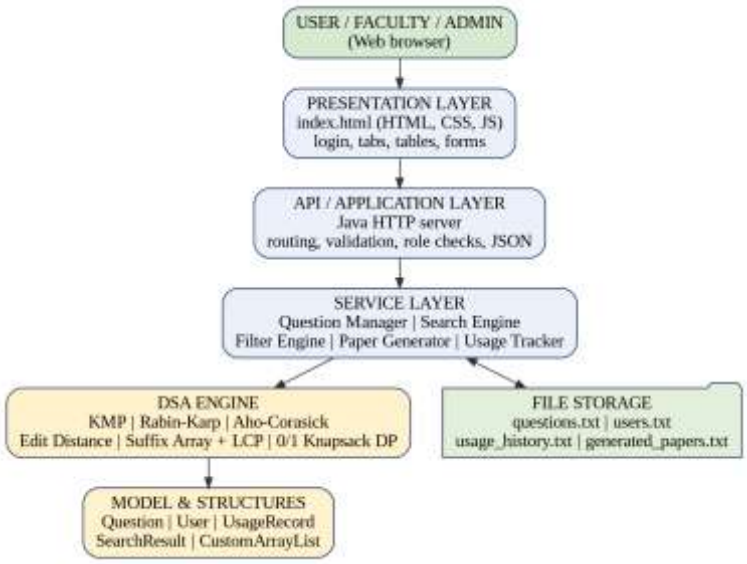


Fig. 3.1 High-level layered architecture diagram

3.2 Description of Each Layer

3.2.1 Presentation (Front-end) Layer

The presentation layer is implemented with HTML, CSS and JavaScript in a single-page interface. It provides the login screen, the dashboard counters, the seven functional tabs, filters, forms and result cards. JavaScript manages user interaction and communicates with the back end using HTTP requests; the returned JSON is rendered dynamically. The layer does not implement any DSA algorithm itself; it only collects input and displays output.

3.2.2 API / Application Layer

The API layer is the bridge between the front end and the back end. It receives HTTP requests, identifies the requested operation, reads parameters or the request body, performs basic validation and role checks, calls the appropriate service method, formats the result as JSON and returns it. This prevents the front end from accessing algorithm or storage code directly.

3.2.3 Service Layer

The service layer holds the application logic. Its major components are the Question Manager (add, edit, delete, list), the Search Engine (query normalisation and algorithm selection), the

Filter Engine (subject, unit, difficulty, CO and marks), the Paper Generator (knapsack-based selection) and the Usage Tracker (history updates). The service layer decides which algorithm should be used for the requested operation, as shown in Fig. 3.2.

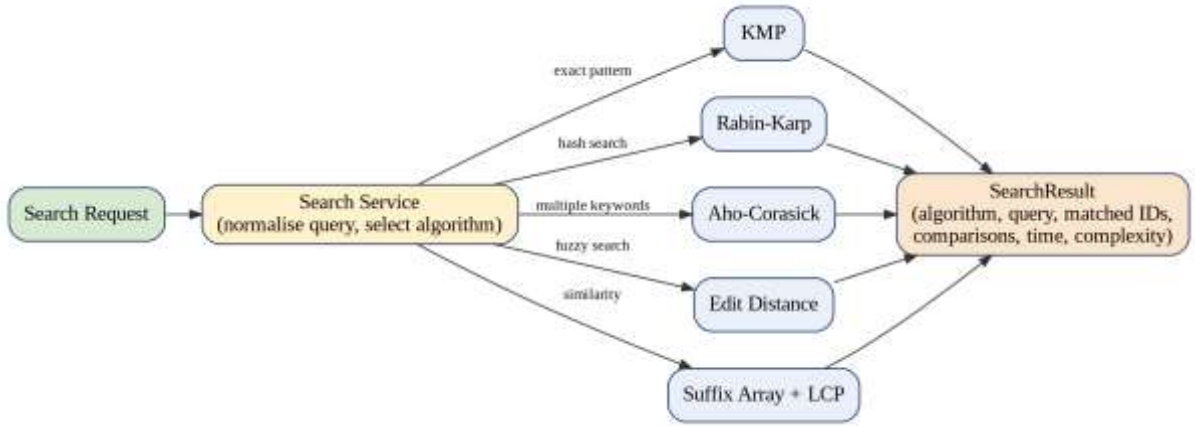


Fig. 3.2 Search request architecture and algorithm routing

3.2.4 Algorithm Layer

The algorithm layer is the core DSA component. Each algorithm is an independent class so that it can be developed, tested and analysed separately from the user-interface code. The algorithm components are:

- **KMP** - exact single-pattern matching using an LPS (longest proper prefix which is also a suffix) array. Application: exact question-text search.
- **Rabin-Karp** - polynomial rolling hash to locate candidate matches. Application: hash-assisted search and algorithm comparison.
- **Aho-Corasick** - trie with failure links for simultaneous multi-pattern matching. Application: searching several keywords in one pass.
- **Edit Distance (Wagner-Fischer)** - dynamic programming over insertions, deletions and substitutions. Application: fuzzy matching of misspelled queries.
- **Suffix Array + LCP** - sorted suffixes and longest common prefixes of neighbours. Application: similarity and duplicate-question detection.
- **0/1 Knapsack DP** - optimal subset selection under a capacity limit. Application: question-paper generation with a target total marks.

3.2.5 Data (File Storage) Layer

The data layer manages persistent information in local files instead of a relational or NoSQL database. Questions, users, usage records and generated papers are stored in separate files. The layer loads records into custom in-memory structures, parses stored lines into objects, writes new or updated records back, and provides data to the service layer. Because of this separation, the algorithm modules never need to know how questions are physically stored.

3.3 Data Flow Architecture

The general data flow of QBMS is shown in Fig. 3.3. The flow ensures that the user interface remains independent of the internal implementation of each algorithm.

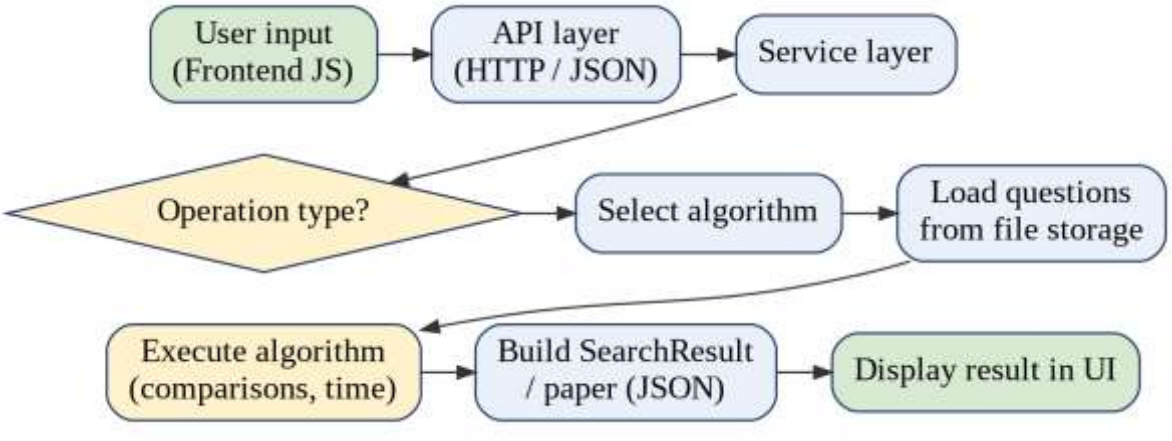


Fig. 3.3 Data flow architecture of QBMS

3.4 Algorithm Selection Architecture

A major design feature of QBMS is that different algorithms are used for different computational requirements, as summarised in the following table.

User Requirement	Algorithm	Primary Technique
Search one exact pattern	KMP	Prefix-function (LPS) based string matching
Hash-based pattern search	Rabin-Karp	Rolling hash
Search multiple keywords	Aho-Corasick	Trie + failure links
Find similar terms (typos)	Edit Distance	Dynamic programming
Compare / de-duplicate questions	Suffix Array + LCP	Suffix structures
Generate a balanced paper	0/1 Knapsack	Dynamic programming with usage penalty

3.5 Component Interaction

The main components interact as shown in Fig. 3.4. The front end calls the HTTP server, which delegates to the service layer. The service layer uses algorithm modules for computation and data services for file access, and the combined result is returned to the front end.

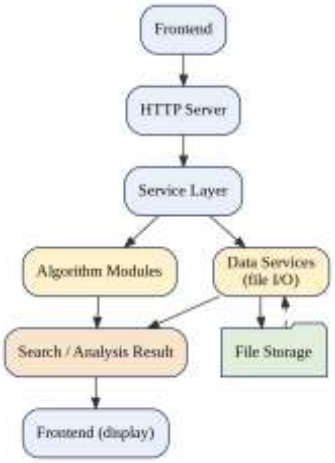


Fig. 3.4 Component interaction diagram

3.6 Data Storage Design

QBMS uses file-based persistence, organised as shown in Fig. 3.5. Each record is stored on one line with fixed fields separated by a safe delimiter such as the pipe character (|). Java File I/O loads these records into custom in-memory structures for algorithmic processing.

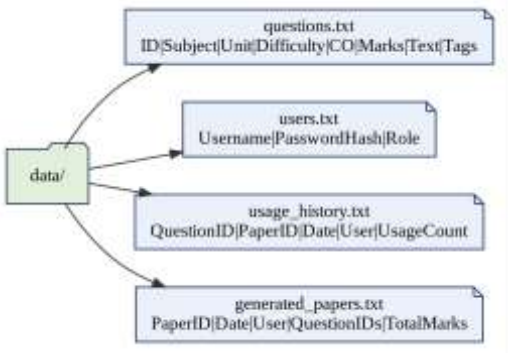


Fig. 3.5 File-based data storage design

File	Record format	Purpose
questions.txt	ID | Subject | Unit | Difficulty | CO | Marks | Question Text | Tags	Master question bank
users.txt	Username | Password hash | Role	Admin and Faculty accounts
usage_history.txt	Question ID | Paper ID | Date | User | Usage Count	Question usage tracking
generated_papers.txt	Paper ID | Date | User | Question IDs | Total marks	Generated paper archive

3.7 Class Design

The principal classes are shown in Fig. 3.6. The Question class acts as the connection between structured academic data and the string algorithms by producing a combined searchable text. SearchResult is an in-memory reporting object that holds the algorithm name, query, matched question IDs, comparison count, execution time and complexity. The AhoCorasick class and the other algorithm classes use the project's own CustomArrayList instead of the Java collections framework.

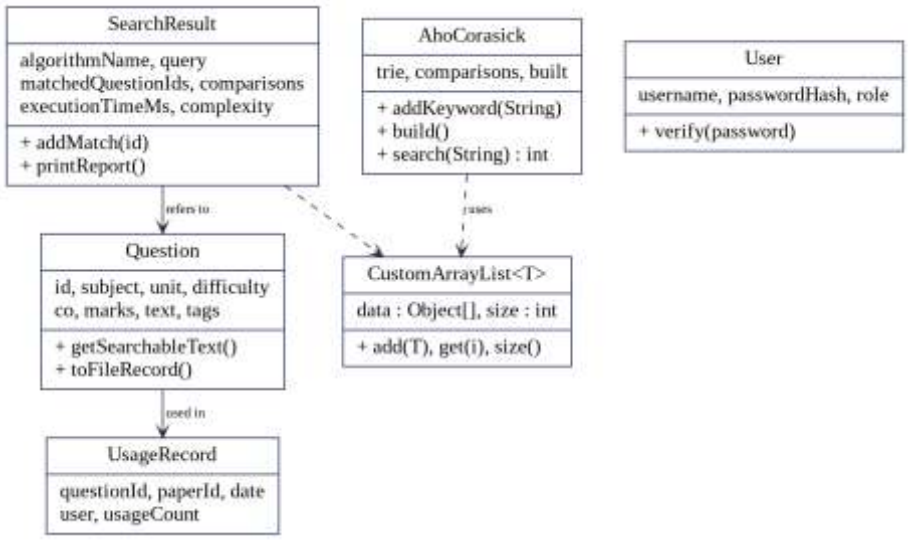


Fig. 3.6 Class diagram of the major model and algorithm classes

3.8 Security and Session Architecture

Users provide credentials through the login screen. The back end validates them against the users file, compares password hashes instead of plain text, and creates a session carrying the user's role (ADMIN or FACULTY). Protected operations check the role before running; for example, user management and benchmarking are available to the Admin. Authentication remains separate from the algorithm layer, so the algorithms operate only on authorised requests. The authentication is deliberately lightweight, as noted in the limitations in Chapter 6.

3.9 Design Principles

- **Modularity** - every algorithm and major function is a separate module.
- **Separation of concerns** - front end, API, business logic, algorithms and storage have distinct responsibilities.
- **Extensibility** - new algorithms can be added to the algorithm layer with minimal changes elsewhere.
- **Algorithmic transparency** - algorithms are implemented directly, not hidden behind external libraries, because the project is meant to demonstrate understanding of DSA concepts.

3.10 Overall System Workflow

The complete workflow, from opening the application to displaying a result, is summarised in Fig. 3.7.

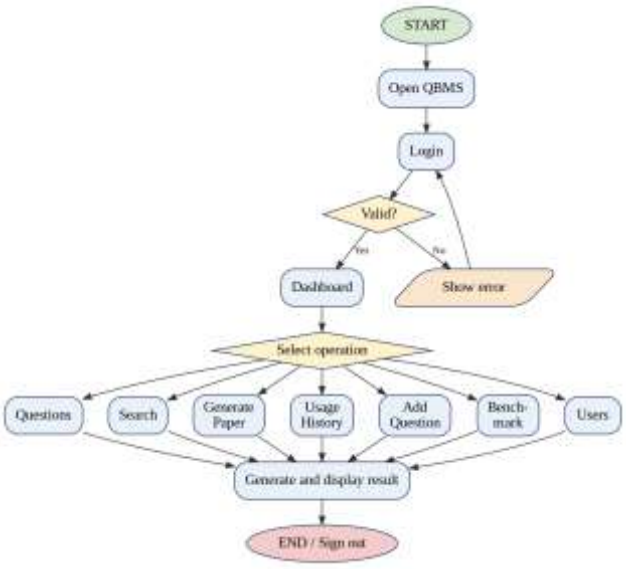


Fig. 3.7 Overall system workflow

4. IMPLEMENTATION

This chapter describes the implementation of QBMS: the back-end modules, the model and data-structure classes, the DSA components, the front-end logic, API communication and file storage. The user interface communicates with the Java back end through HTTP, while the service layer coordinates question operations and algorithm execution. The Java source is organised into packages such as algorithms, model and structures, and the front end is maintained in a separate QBMS-Frontend project.

4.1 Module-wise Implementation

4.1.1 Server Module

The server module is the entry point of the Java application. It creates and starts the HTTP server, loads the data files into memory, registers the request handlers and then waits for requests from the front end, as shown in Fig. 4.1. The handlers cover login and logout, question retrieval and modification, searching, paper generation, usage history, user management and delivery of the static front-end files. A lightweight built-in server is used rather than a framework, which keeps the back end small and lets the project concentrate on the DSA components.

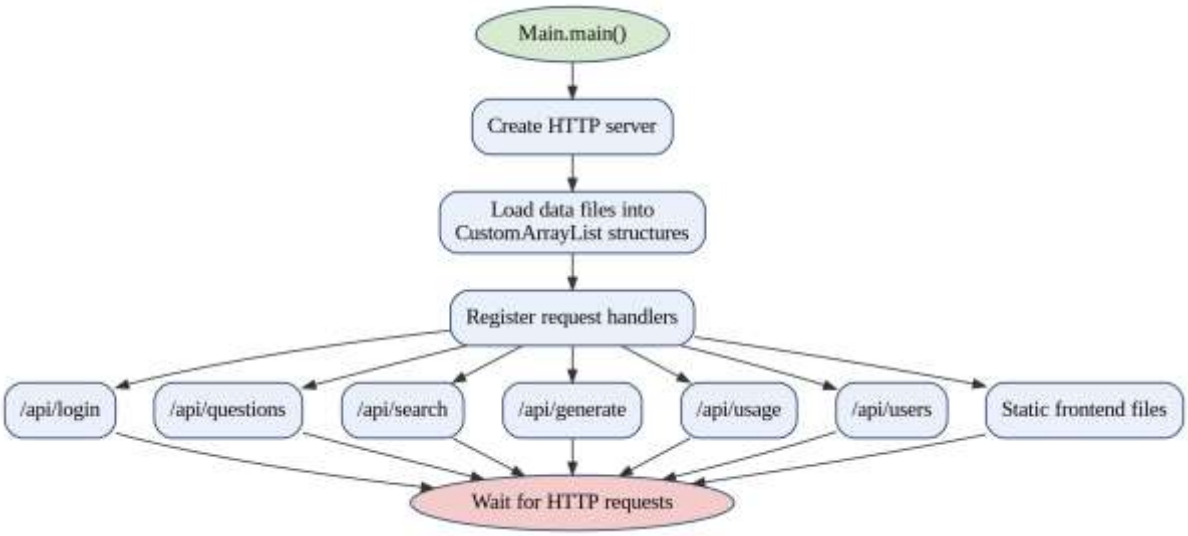


Fig. 4.1 Server start-up and request handler registration

4.1.2 Model Module

The model package contains the classes that represent application data. The **Question** class stores the ID, subject, unit, difficulty, CO, marks, question text and tags, and can produce a combined, lower-cased searchable text as in Fig. 4.2. The **User** class stores the username, password hash and role. The **UsageRecord** class stores the question ID, paper ID, date, user and usage count.



Fig. 4.2 Question object converted into searchable text

The **SearchResult** class (source shown in Fig. 4.3) holds the outcome of running a search algorithm so that the interface and the benchmark module can display a uniform report: Algorithm, Query, Matches, Comparisons, Execution Time and Complexity. It is an in-memory reporting object and is not persisted to a file. It exposes getters, an addMatch method to collect

matched question IDs, setters for comparisons, execution time and complexity, and a printReport method.



Fig. 4.3 Source code of the SearchResult model class

4.1.3 Custom Data Structures

The structures package provides the project's own **CustomArrayList** generic class. It wraps a growing array with add, get and size operations and is used instead of java.util collections for storing questions, matched IDs and the trie nodes of Aho-Corasick. Implementing the structure manually reinforces the DSA focus of the project and allows the number of element accesses to be reasoned about directly.

4.1.4 File Storage Service

The file service is responsible for persistence and retrieval. It performs the following operations: loading all questions from questions.txt, parsing each line into a Question object, appending newly added questions, rewriting the file after an edit or a delete, generating the next question ID, and appending to the usage-history and generated-papers files. Because the algorithms receive question text rather than opening files themselves, the data layer stays independent of the search code.

4.1.5 User and Authentication Module

The authentication logic is divided between the User model, the user file service and the login handler. The password entered by the user is hashed and compared with the stored hash, and on success a session carrying the role is created. On failure the interface shows an error message with a short shake animation. The login process is shown in Fig. 4.4.

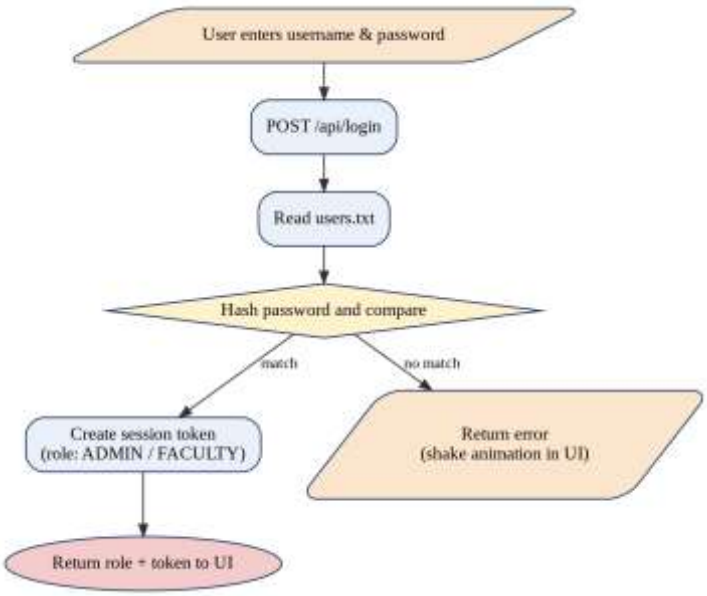


Fig. 4.4 Login and authentication flow

4.1.6 Question Management Module

Faculty add a question through the Add Question tab. The handler validates the fields, creates the Question object with the next generated ID, runs a similarity check using Suffix Array and LCP, appends the record to questions.txt and updates the in-memory list and dashboard counters (Fig. 4.5). Each question card in the Questions tab also provides edit and delete buttons, and the list can be filtered by subject, unit and difficulty using the filter box and drop-downs.

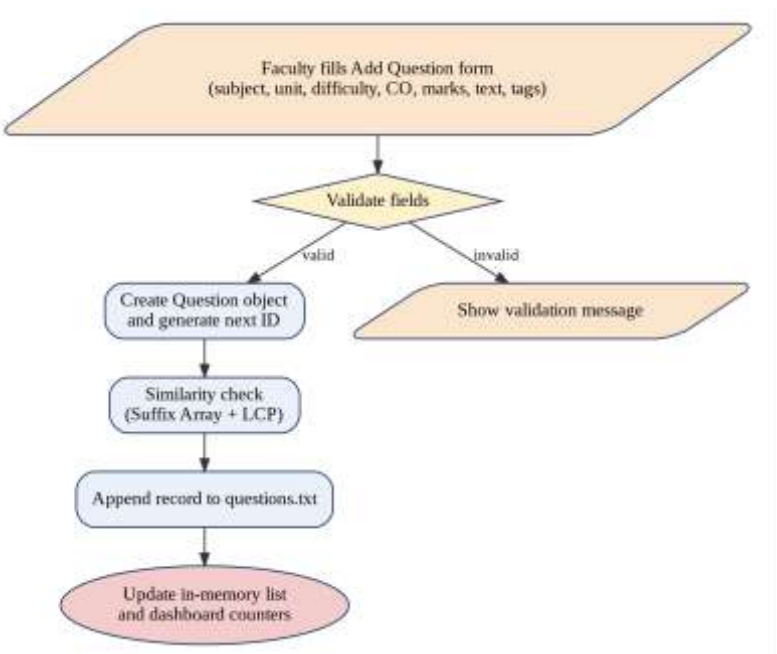


Fig. 4.5 Add-question flow with validation and similarity check

4.1.7 Search Service Module

The search service is the central business-logic component for retrieval. It receives the query and the chosen algorithm, normalises the input (lower-casing, trimming and splitting keywords), removes duplicate keywords, runs the selected algorithm over the searchable text of every question, and collects matched IDs, comparison counts and timing in a SearchResult. When no

exact match is found, edit distance is used to suggest close matches. The flow is shown in Fig. 4.6.

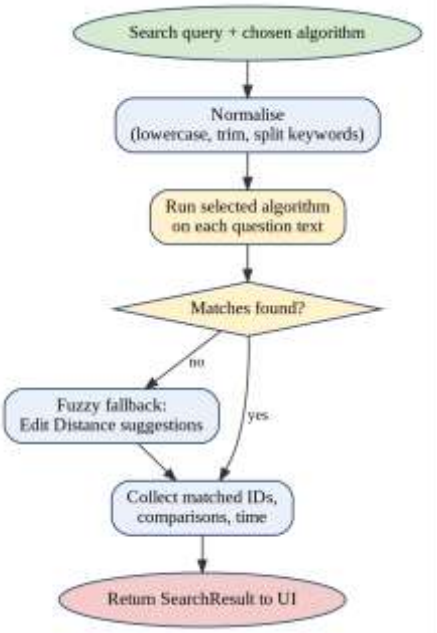


Fig. 4.6 Search service flow

4.1.8 Aho-Corasick Module

The AhoCorasick class implements multi-pattern string matching. As its header comment states, the algorithm builds a trie from all pattern keywords, computes failure links through a breadth-first traversal (like the KMP failure function but on a trie) and scans the text once, following goto and failure transitions to find all patterns simultaneously. Its complexity is O(n + sum of pattern lengths + total matches), and it returns the number of keyword matches found in the text. The class stores trie nodes in a CustomArrayList; each Node holds an int array of 26 children (initialised to -1), a failLink, an isEnd flag and the keyword that ends there. It also keeps a comparison counter and a built flag so that the trie is constructed once and reused. Fig. 4.8 shows the source, and Fig. 4.7 illustrates a trie with failure links (dashed) for the classic patterns he, hers, his and she.

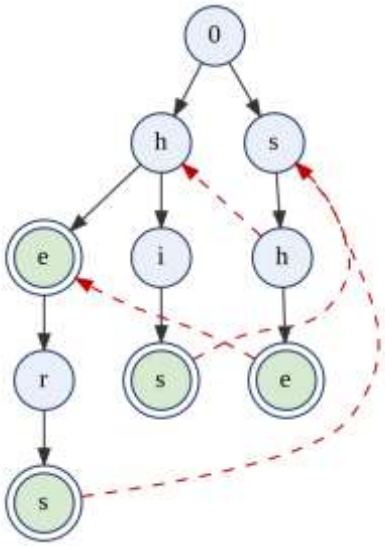


Fig. 4.7 Aho-Corasick trie with failure links (patterns: he, hers, his, she)

```java
package algorithms;

import structures.CustomArrayList;

/*
 * Algorithm: Aho-Corasick multi-pattern string matching automaton
 * Complexity: O(n + Σ|patterns| + total_matches)
 *
 * How it works:
 *  1. Build a Trie from all pattern keywords.
 *  2. Compute failure links via BFS (like KMP failure function but on a Trie).
 *  3. Scan the text once; follow goto/failure transitions to find all patterns
 *     simultaneously in a single pass.
 *
 * Returns the number of keyword matches found in the text.
 */
public class AhoCorasick {

    // ---- Trie node ----
    private static class Node {
        int[] children;
        int failLink;
        boolean isEnd;
        String keyword;

        Node() {
            children = new int[26];
            java.util.Arrays.fill(children, -1);
            failLink = 0;
            isEnd = false;
            keyword = null;
        }
    }

    private CustomArrayList<Node> trie;
    private long comparisons;
    private boolean built;

    public AhoCorasick() {
        trie = new CustomArrayList<>();
        trie.add(new Node()); // root = node 0
```

Fig. 4.8 Source code of the AhoCorasick algorithm class

4.1.9 KMP Module

The Knuth-Morris-Pratt algorithm performs exact pattern matching in O(n + m) time by pre-processing the pattern into an LPS array. On a mismatch the pattern index falls back using the LPS value so that the text pointer never moves backwards; this avoids the repeated comparisons of naive matching. It is the primary exact-search algorithm of QBMS and is especially useful for repeated searches with the same pattern. The flow is shown in Fig. 4.9. For the pattern ababaca the LPS array is shown below.

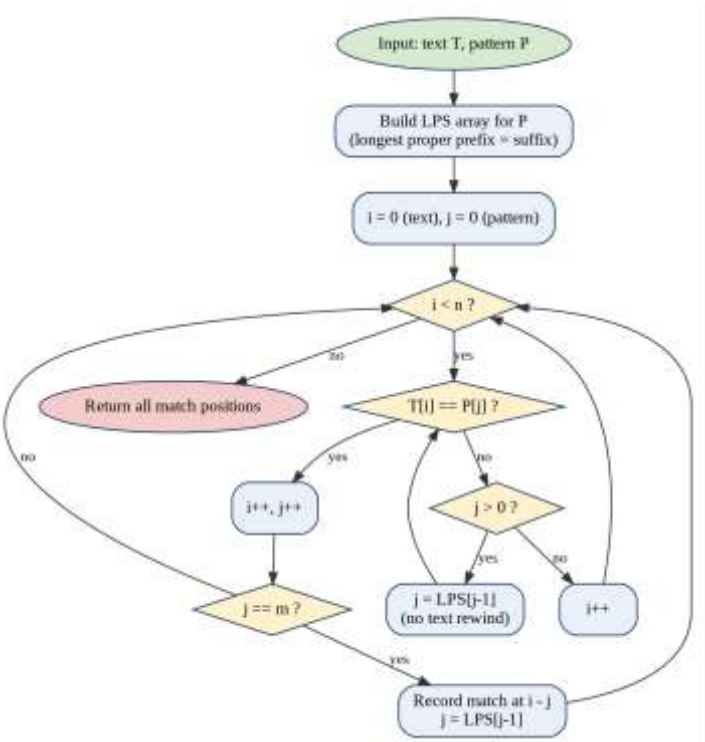


Fig. 4.9 Flowchart of the KMP algorithm

Index	0	1	2	3	4	5	6
Pattern	a	b	a	b	a	c	a
LPS	0	0	1	2	3	0	1

4.1.10 Rabin-Karp Module

Rabin-Karp uses polynomial hashing. The hash of the pattern is compared with the hash of every window of the text; the window hash is updated in O(1) by removing the leading character and adding the trailing one (a rolling hash), using h = (d * (h - T[i] * d^(m-1)) + T[i+m]) mod q for a base d and prime modulus q. Only when hashes match are the characters compared, which rules out collisions. The average-case cost is O(n + m); the worst case is O(nm). In QBMS it provides hash-assisted search and is useful when several query patterns are checked against the same text. The flow is shown in Fig. 4.10.

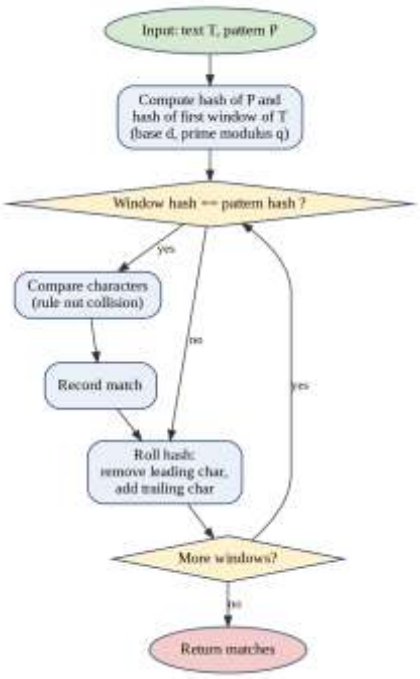


Fig. 4.10 Flowchart of the Rabin-Karp algorithm

4.1.11 Edit Distance Module

Fuzzy search is implemented with the Wagner-Fischer dynamic-programming algorithm. For a query s of length n and a bank term t of length m, a table dp[i][j] stores the minimum number of insertions, deletions and substitutions needed to convert the first i characters of s into the first j characters of t, using dp[i][j] = min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + cost). The time and space cost is O(nm). Terms whose distance is within a threshold are suggested to the user, so a query such as "stak" still finds questions about "stack". Fig. 4.11 shows the flow and the table below shows the DP matrix for converting "stak" to "stack" (distance = 1).

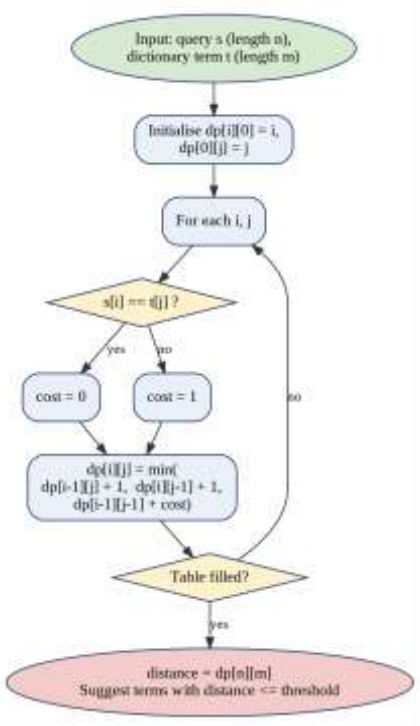


Fig. 4.11 Flowchart of Wagner-Fischer edit distance

		s	t	a	c	k
	0	1	2	3	4	5
s	1	0	1	2	3	4
t	2	1	0	1	2	3
a	3	2	1	0	1	2
k	4	3	2	1	1	1

4.1.12 Suffix Array and LCP Module

To detect similar or repeated questions, the question texts are concatenated with separators, all suffixes are generated and sorted lexicographically to form the suffix array, and the LCP array stores the longest common prefix between adjacent suffixes. A large LCP value between suffixes that belong to different questions reveals a long common substring, so the new question is flagged as similar (Fig. 4.12). Construction cost depends on the sorting method used, and the LCP array can be computed in linear time with Kasai's approach.

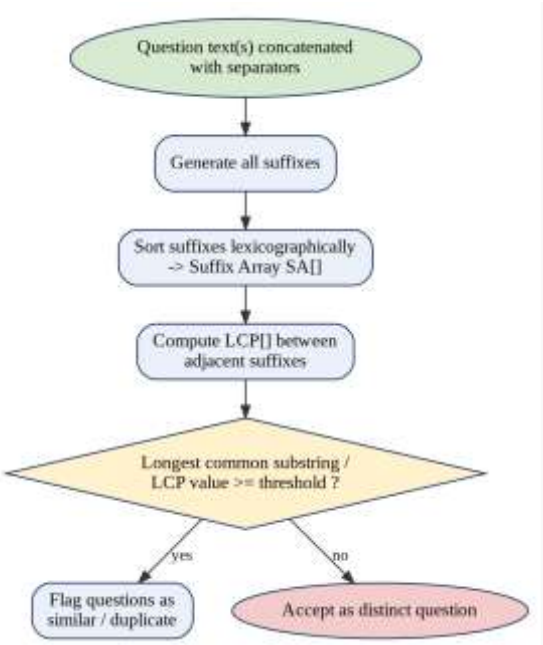


Fig. 4.12 Suffix Array and LCP based similarity detection

4.1.13 Paper Generator Module

The Generate Paper tab takes three inputs: the subject (or all subjects), the number of questions and the target total marks. The interface describes the constraint as "0/1 Knapsack DP selects the optimal question set; overused questions are penalised". Implementation-wise, the candidate questions are filtered by subject; each question receives a score that is reduced according to its usage count in the history file; the DP table is built with the question marks as item weight, the score as item value and the target marks as capacity, limited to the requested number of questions; and the table is back-tracked to recover the chosen questions (Fig. 4.13). The recurrence is dp[i][w] = max(dp[i-1][w], dp[i-1][w-marks_i] + score_i), with time complexity O(N x W) for N candidates and capacity W (an extra dimension for the question count).

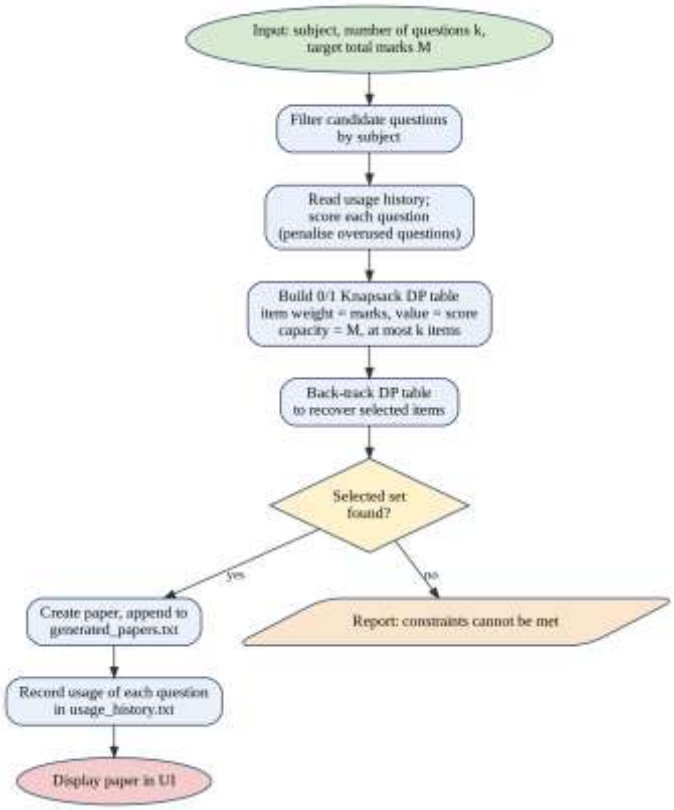


Fig. 4.13 Paper generation using 0/1 Knapsack dynamic programming

Question	Marks (weight)	Score (value)	Remark
Q1	5	6	Fresh question
Q2	10	9	Fresh question
Q3	10	4	Overused - score reduced
Q4	5	5	Fresh question
Target = 20 marks, max 3 questions			Selected: Q1, Q2, Q4 (total score 20)

4.1.14 Usage Tracker Module

After a paper is generated, the usage tracker updates usage_history.txt for every selected question: if a record exists its usage count is incremented, otherwise a new record is created (Fig. 4.14). The dashboard counters "Papers Generated" and "Usage Records" are refreshed, and the Usage History tab lists the records. Because the paper generator reads the same history, frequently used questions are gradually penalised, which reduces repetition over time.

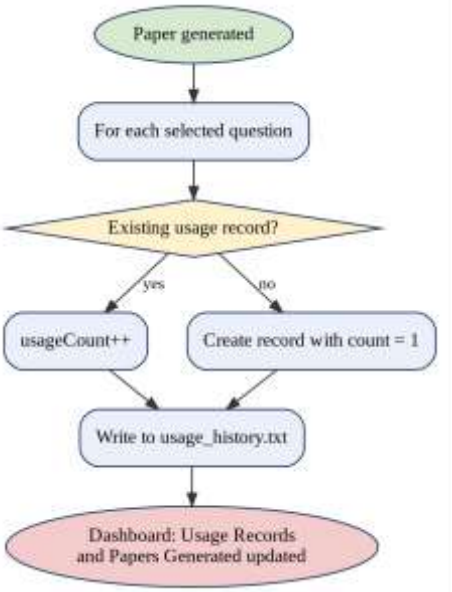


Fig. 4.14 Usage tracking flow

4.1.15 Benchmark Module

The Benchmark tab compares the algorithms on the same question bank. Each run creates a SearchResult containing the algorithm name, the query, the matched question IDs, the number of character comparisons, the execution time in milliseconds and the theoretical complexity string. This supports the project objective of proving algorithmic improvement and not just functionality, for example by comparing the character comparisons of naive matching against KMP on the same data.

4.2 Front-end Logic

4.2.1 UI Rendering

The front end is a single index.html file with embedded CSS and JavaScript. The page contains a login screen and a dashboard with four statistic cards and a tab bar. JavaScript renders question cards with their subject, unit, difficulty, CO, marks and tag chips, and shows search results and generated papers dynamically without reloading the page. The look is a dark glass-morphism theme with blurred translucent cards, gradient accent colours and animated background blobs, as in Fig. 4.15. A Theme button switches between dark and light modes.

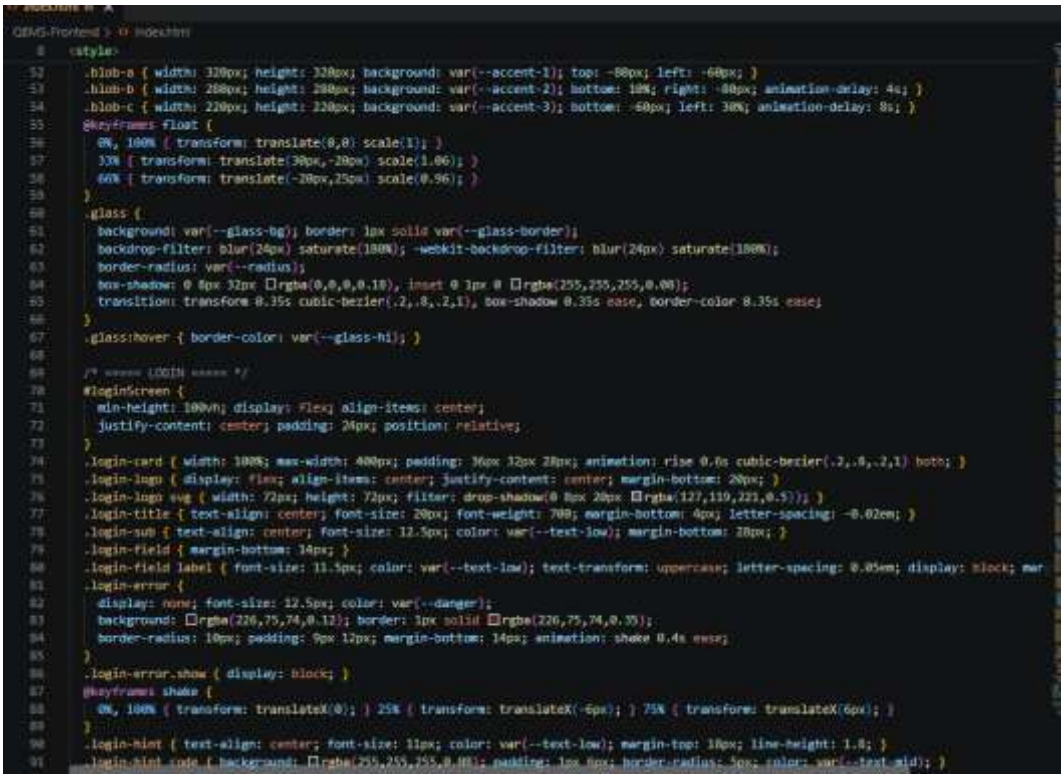


Fig. 4.15 Front-end styling code (glass-morphism theme) in index.html

4.2.2 State Management

The front end keeps simple application state in JavaScript variables: the signed-in user and role, the loaded question list, the active tab, the current filters, and the latest search or generation result. Changes to this state re-render only the affected part of the interface, so the page updates without a full reload. Role information decides which tabs and buttons are shown.

4.2.3 User Interaction

Users sign in, switch between tabs, filter questions by text, difficulty and subject, run searches, add or edit questions, generate papers and review usage history. Validation errors and login failures are displayed inline, and hover and transition effects give immediate feedback.

4.3 API Integration

The front end talks to the Java back end through HTTP endpoints and exchanges JSON. The main interactions are summarised below.

API Endpoint	Method	Purpose
/api/login	POST	Authenticates the user and returns role and session
/api/questions	GET / POST / PUT / DELETE	List, add, edit and delete questions
/api/search	POST	Runs the chosen search algorithm and returns a SearchResult
/api/generate	POST	Generates a question paper using knapsack DP
/api/usage	GET	Returns usage-history records
/api/benchmark	POST	Runs the algorithm benchmark

API Endpoint	Method	Purpose
/api/users	GET / POST	Lists and manages users (Admin)

Requests are issued from JavaScript and the JSON responses are processed to update the interface, which keeps the user interface separate from the application logic and lets the algorithms operate independently of the front end.

4.4 Data Processing Flow

The basic processing flow of QBMS is: User Input - Front end - HTTP API - Service Layer - Selected Algorithm - Question Records - SearchResult / Paper - Front end. The service layer loads the questions from file storage into a CustomArrayList, the algorithm processes their searchable text while counting comparisons, and the result is returned as JSON for display.

4.5 Error Handling

The system validates login credentials, question fields, search queries and paper-generation inputs. Invalid logins show an error message; empty or unmatched searches return an empty result instead of failing; missing questions are reported to the interface; and when the paper constraints cannot be satisfied (for example, a target total that the available questions cannot meet) the user is informed instead of receiving an incorrect paper.

4.6 Implementation Summary

QBMS follows a layered structure in which the front end handles interaction, the Java back end manages requests and logic, the algorithm layer provides retrieval, similarity and selection, and file services provide persistence. The algorithms are implemented manually in the algorithms package, and the whole application can be run with just a Java runtime and its data files.

5. RESULTS

5.1 Functional Results

The developed QBMS integrates the web front end with the Java back end and provides the following working functionality:

- Role-based login for Admin and Faculty users.
- Display of the question bank (56 questions across 5 subjects in the demonstration data) with subject, unit, difficulty, CO, marks and tags.
- Filtering by text, difficulty and subject, and edit/delete controls on every question.
- Algorithm-driven search with a report of matches, comparisons, execution time and complexity.
- Paper generation for a chosen subject, number of questions and target total marks using 0/1 Knapsack DP.
- Usage-history tracking and live dashboard counters.
- Benchmark and user-management tabs for administrators.

5.2 User Interface Results

The following screenshots of the working application show the main screens.



Fig. 5.1 Login page of QBMS

The login page (Fig. 5.1) shows the application title, the subtitle "DSA-Driven - File Storage - Java Backend", username and password fields and a Sign in button.

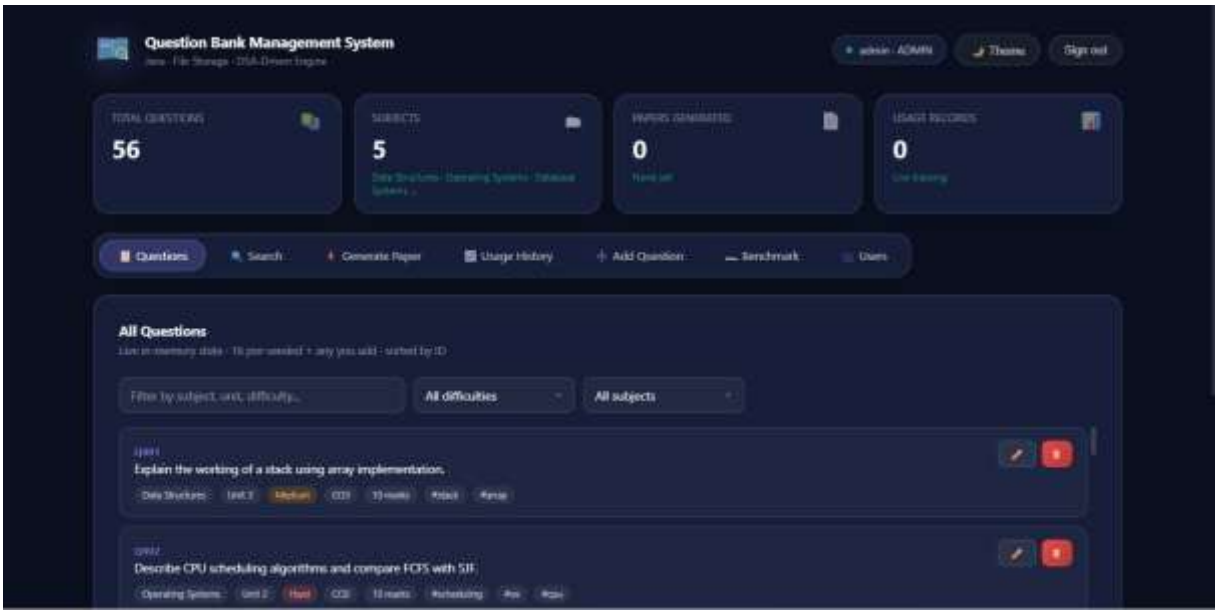


Fig. 5.2 Dashboard with statistics and the Questions tab

After signing in as admin, the dashboard (Fig. 5.2) shows the signed-in user and role, the Theme and Sign out buttons, and four counters (Total Questions 56, Subjects 5, Papers Generated, Usage Records). The Questions tab lists each question with its ID, text and metadata chips.

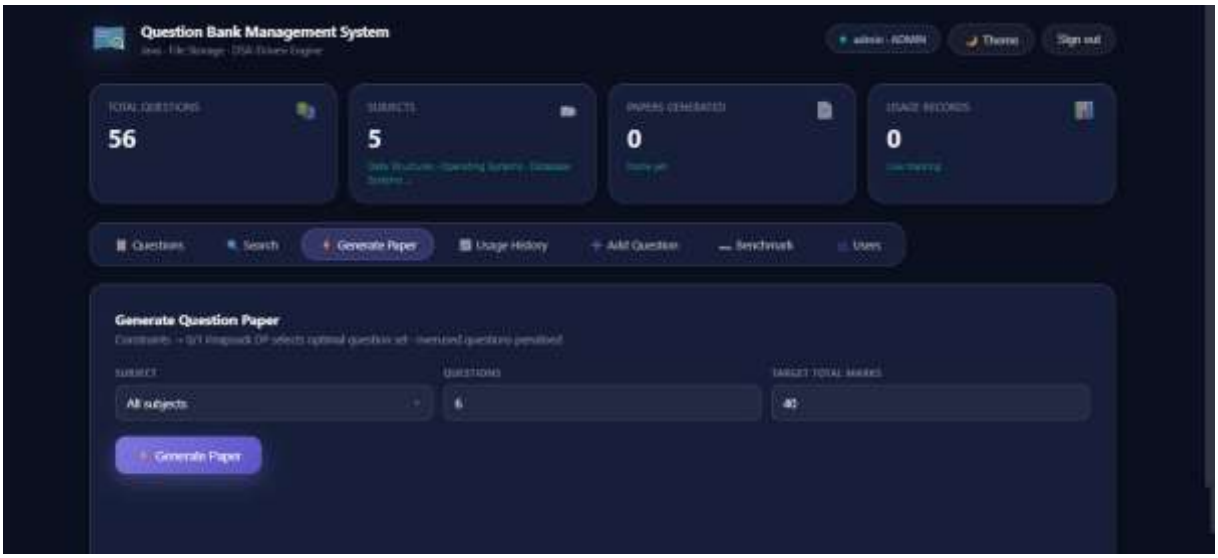


Fig. 5.3 Generate Paper tab with subject, question count and target marks

The Generate Paper tab (Fig. 5.3) accepts the subject, the number of questions (6) and the target total marks (40), and generates the paper using the 0/1 Knapsack constraint with overused questions penalised.

5.3 Algorithm Complexity Comparison

The algorithms used in QBMS differ in cost and purpose. The table compares their theoretical behaviour; the Benchmark tab measures the practical comparison counts and times on the stored question bank.

Technique	Typical cost	Space	Role in the project
Naive matching	O(nm)	O(1)	Baseline for comparison
KMP	O(n + m)	O(m)	Primary exact search

Technique	Typical cost	Space	Role in the project
Rabin-Karp	Average O(n + m), worst O(nm)	O(1)	Hash-assisted search
Aho-Corasick	O(n + sum|P| + z)	O(sum|P|)	Multi-keyword search
Edit Distance	O(nm)	O(nm)	Fuzzy matching
Suffix Array + LCP	Construction-dependent; LCP O(n)	O(n)	Similarity / duplicate analysis
0/1 Knapsack DP	O(N x W)	O(N x W)	Constraint-based paper generation

5.4 Test Results

The following test cases were used to evaluate the implemented functionality.

Test Case	Input	Expected Result	Actual Result	Status
TC01	Valid login credentials	User signs in and dashboard opens	Signed in successfully	Pass
TC02	Invalid password	Error message shown, access denied	Error shown	Pass
TC03	Open Questions tab	All questions displayed with metadata	Questions displayed	Pass
TC04	Filter by difficulty / subject	Only matching questions shown	Filtered list shown	Pass
TC05	Exact keyword search	Matching questions returned	Matching questions returned	Pass
TC06	Multiple keyword search	Questions containing any keyword returned	Matching questions returned	Pass
TC07	Misspelled query	Close matches suggested	Suggestions returned	Pass
TC08	Unknown keyword	No matching questions returned	Empty result	Pass
TC09	Empty search input	Handled without system failure	Input handled	Pass
TC10	Add a new question	Question saved and counters updated	Question added	Pass
TC11	Edit / delete a question	File and list updated	Updated correctly	Pass
TC12	Generate paper (6 questions, 40 marks)	Paper meeting the constraints generated	Paper generated	Pass
TC13	Usage history after generation	Usage records created	Records created	Pass

5.5 Overall Result

The implemented QBMS demonstrates that advanced DSA-3 techniques can be combined into a practical question-bank application. The front end, Java back end, file-based storage and algorithm package work together to retrieve relevant questions, tolerate misspelled queries, detect similar questions and generate constraint-aware question papers.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

- Designing the algorithm layer so that every algorithm returns comparable results (comparisons, time and complexity) for the benchmark.
- Implementing data structures manually (CustomArrayList and trie nodes) instead of using ready-made collections.
- Handling file reading, writing, rewriting after edits and ID generation without a database.
- Integrating the front end with the Java back end through HTTP and JSON.
- Choosing a scoring scheme for the knapsack so that overused questions are penalised without making the paper unbalanced.
- Validating invalid, empty or unmatched inputs without breaking the application.

6.2 Solutions Applied

- The system was divided into separate packages: algorithms, model, structures, services and the front end.
- A common SearchResult model was introduced to report algorithm name, matches, comparisons, execution time and complexity uniformly.
- A reusable CustomArrayList and an array-based trie with fail links were implemented and reused across modules.
- Records are stored one per line with a safe delimiter, which makes parsing and rewriting predictable.
- The knapsack score is reduced by usage count, and the DP table is back-tracked to obtain the exact set of questions.
- Input validation and basic error messages were added to every handler and to the interface.

6.3 Limitations of the System

- File storage is not ideal for concurrent multi-user production systems.
- Large-scale indexing may require careful memory management.
- Perfectly balanced paper generation can become computationally expensive for very large banks.
- Authentication is lightweight because no database is used.
- The system is an academic project and does not include exam-scheduling, approval workflows or analytics.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Question Bank Management System turns a common academic workflow into a practical DSA-3 problem. Java handles the application, files handle persistence, and advanced algorithms handle the intelligence: KMP, Rabin-Karp and Aho-Corasick for string search, edit distance for fuzzy matching, Suffix Array with LCP for similarity, and 0/1 Knapsack dynamic programming for paper generation. The project provides practical experience in algorithm implementation, custom data structures, back-end development, API communication, file handling and front-end design, and establishes a foundation for further extension.

7.2 Future Work

- Move persistence to a database and deploy the system as a web service as a separate engineering layer.
- Add parallel search for very large repositories.
- Introduce stronger randomised hashing for indexing.
- Use network-flow based allocation for complex paper constraints (unit, difficulty and CO balance together).
- Add an analytics dashboard for question usage and quality.
- Improve authentication and security.
- Add export of generated papers to PDF and Word formats.

7.3 Skills Learned During Development

- Data Structures and Algorithms (string algorithms, dynamic programming, complexity analysis)
- Java programming and file handling
- Custom data-structure implementation
- Backend and API development
- HTML, CSS and JavaScript
- Debugging and testing
- Software architecture and modular design
- Technical documentation and project presentation

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
2. J. Kleinberg and E. Tardos, Algorithm Design. Boston, MA, USA: Pearson, 2005.
3. D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.
4. R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.
5. A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.
6. R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” Journal of the ACM, vol. 21, no. 1, pp. 168–173, 1974.
7. U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” SIAM Journal on Computing, vol. 22, no. 5, pp. 935–948, 1993.
8. T. Kasai et al., “Linear-time longest-common-prefix computation in suffix arrays and its applications,” in Proc. CPM, 2001, pp. 181–192.
9. Oracle, “Java SE Documentation.” [Online]. Available: https://docs.oracle.com/en/java/
10. MDN Web Docs, “HTML, CSS and JavaScript Documentation.” Mozilla. [Online]. Available: https://developer.mozilla.org/
11. OpenJDK, “Java Platform, Standard Edition Documentation.” [Online]. Available: https://openjdk.org/